

Universidad Mariano Galvez de Guatemala



“Análisis, Rediseño Arquitectónico y Contrato API”

Curso: Análisis de sistemas II

Integrantes

Jorge Estuardo Perez Vasquez

Contenido

Descripción del problema.....	3
Informe de Ingeniería Inversa y Rediseño Arquitectónico.....	3
Contexto del Sistema Legado	3
Funcionalidades principales.....	3
Diagrama del sistema heredado	4
Matriz Comparativa de Arquitecturas.....	5
Especificación y Contrato OpenAPI 3.0 (Swagger).....	8
Códigos de estado HTTP	9
Seguridad	9
Procesos de autenticación	9
Autorización mediante roles y permisos	10
Planos Técnicos de Arquitectura bajo el Modelo C4	11
Nivel 1 – Diagrama de contexto	11
Nivel 2 – Diagrama de contenedores	12
EDR.....	13
Estrategia de Caché	13
Claves de Caché y TTL.....	14
Política de invalidación ante mutaciones.....	15

Descripción del problema

Actualmente, los procesos de votación se realizan de forma manual, lo cual genera problemas como:

- Errores en conteo de votos
- Retrasos en la consolidación de resultados
- Falta de control en mesas electorales
- Dificultad para auditar resultados

Se requiere diseñar e implementar un sistema que permita gestionar el proceso de votación de manera ordenada, rápida y confiable.

Informe de Ingeniería Inversa y Rediseño Arquitectónico

Contexto del Sistema Legado

El sistema de votaciones fue desarrollado originalmente como una aplicación web monolítica utilizando PHP.

El entorno tecnológico principal está compuesto por:

- Lenguaje: PHP
- Servidor web: Apache
- Entorno de ejecución: XAMPP
- Base de datos: MySQL / MariaDB
- Interfaz: HTML, CSS y navegador web
- Sistema de gestión de base de datos: votaciones_db
- Tipo de arquitectura: aplicación web monolítica
- Despliegue inicial: Servidor local

El sistema se ejecuta originalmente dentro de un solo equipo de manera local donde accede por medio de la dirección: http://localhost/sistema_votaciones/login.php

La aplicación contiene diferentes paginas PHP las cuales son responsables tanto de presentar información al usuario como de procesar las operaciones y acceder a la base de datos.

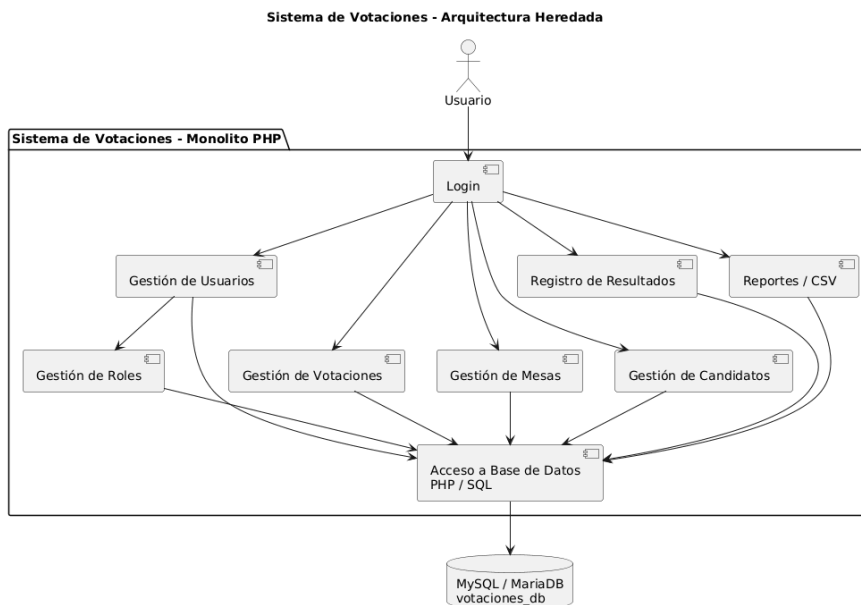
Funcionalidades principales

El análisis del sistema permite identificar distintos módulos, entre ellos están:

- **Gestión de autenticación:** Lo cual permite que un usuario pueda iniciar sesión utilizando sus credenciales y acceder a las funciones asignadas según su rol. Entre la aplicación se encuentran roles como Administrador, Operación, digitación y consulta
- **Gestión de usuarios:** Permite administrar los usuarios que pueden ingresar al sistema y asignar los roles correspondientes, las acciones principales entre los roles son: Crear usuarios, consultar usuarios, modificar usuarios y gestionar roles.
- **Gestión de votaciones:** Permite crear y administrar los distintos procesos electorales registrados en el sistema
- **Gestión de mesas:** Permite registrar las mesas que participan en una determinada votación y que posteriormente serán utilizadas para registrar resultados.
- **Gestión de candidatos:** Permite registrar los candidatos asociados con una determinada votación.

Entre estos y mas fue basado el sistema inicial.

Diagrama del sistema heredado



El diagrama evidencia que todos los módulos dependen del mismo componente de acceso a datos y de una única base de datos, por esta razón es que el sistema se considera que es un sistema monolito, donde cada uno de los componentes forman parte de una misma aplicación y generalmente son desplegados de manera conjunta y que si uno llegase a fallar el sistema completo colapsaría.

Matriz Comparativa de Arquitecturas

Para el rediseño del sistema de votaciones se comparó la arquitectura monolítica heredada con la arquitectura propuesta de Monolito Modular + API REST con una separación de base de datos por responsabilidad.

La propuesta contiene una única aplicación desplegable, pero se divide internamente el sistema en módulos funcionales delimitados y distribuye el almacenamiento de información entre diferentes bases de datos según el tipo de carga y responsabilidad.

Las bases de datos propuestas son:

- Base de datos de identidad: Usuarios, credenciales, roles, permisos y sesiones.
- Base de datos central: Votaciones, candidatos, mesas y resultados.
- Base de datos de reportes: Consultas analíticas, estadísticas, agregaciones y reportes.
- Base de datos de auditoria: Historial de operaciones, eventos y trazabilidad.

Con esto se busca evitar que procesos secundarios compitan directamente por los mismos recursos utilizados en operaciones críticas del proceso electoral.

Arquitectura ▼	Pros ▼	Contras ▼	Riesgos ▼
Monolito tradicional	Desarrollo sencillo, bajo costo inicial, una única aplicación, una sola base de datos, transacciones simples, despliegue sencillo.	Alto acoplamiento, dificultad de mantenimiento, baja separación de responsabilidades, escalabilidad limitada, integración externa complicada, todas las cargas comparten recursos.	Crecimiento desordenado del código, BD convertida en cuello de botella, consultas de reportes afectando el registro de resultados, fallos con impacto global, cambios que generan efectos secundarios.
Monolito Modular + API REST + múltiples BD por responsabilidad	Separación clara de responsabilidades, menor acoplamiento, mejor mantenibilidad, API reutilizable, aislamiento de reportes y auditoría, reducción de competencia sobre la BD principal, seguridad centralizada, facilita evolución futura hacia microservicios.	Mayor complejidad que el monolito tradicional, varios almacenes de datos, requiere mecanismos de sincronización, continúa siendo una única aplicación desplegable, no permite escalamiento independiente de módulos inicialmente.	Acoplamiento entre módulos si no se respetan sus límites, inconsistencia entre bases, crecimiento excesivo de algunos módulos, fallos en sincronización, eventual saturación de Core DB, aumento de complejidad operativa.

Criterio	Monolito tradicional actual	Monolito Modular + API REST propuesto
Estructura general	Toda la lógica, presentación y acceso a datos se encuentran concentrados dentro de una misma aplicación con límites poco definidos.	Continúa siendo una aplicación única, pero organizada mediante módulos con responsabilidades claramente delimitadas.
Organización funcional	Las funcionalidades pueden compartir lógica, consultas y componentes directamente.	Se divide en módulos de Identidad, Control de Acceso, Votaciones, Candidatos, Mesas, Resultados y Reportes.
Punto de acceso	Las páginas PHP pueden acceder directamente a la lógica y base de datos.	Las operaciones se exponen mediante una API REST , estableciendo una interfaz uniforme para clientes web, móviles o externos.
Acoplamiento	Alto. Los componentes pueden depender directamente de otros archivos, consultas SQL o tablas.	Bajo a moderado, siempre que los módulos se comuniquen mediante contratos y no accedan directamente a los repositorios internos de otros módulos.
Separación de responsabilidades	Limitada. Una misma página puede presentar información, validar datos, ejecutar lógica y acceder a la BD.	Alta. Controladores, servicios, repositorios y módulos tienen responsabilidades diferenciadas.
Mantenibilidad	Disminuye conforme aumenta el tamaño de la aplicación.	Alta, debido a que los cambios pueden limitarse al módulo responsable.
Base de datos	Una única base compartida por todas las funcionalidades.	Varias bases de datos organizadas según responsabilidad
Riesgo de cuello de botella en la BD	Alto. Autenticación, operaciones electorales, auditoría y reportes compiten por los mismos recursos.	Menor. Las cargas se distribuyen entre bases especializadas según su responsabilidad.
Autenticación	Puede encontrarse mezclada con otras funciones del sistema.	Se divide en Identidad y Control de Acceso, utilizando una Base de datos de identidad específica.
Seguridad	Los controles pueden encontrarse distribuidos entre distintas páginas.	La API permite centralizar autenticación, autorización, JWT, validaciones, roles y permisos.
Procesamiento asíncrono	Generalmente no existe.	Puede utilizarse para auditoría, reportes, exportaciones y actualización de modelos de lectura.
Consultas pesadas	Pueden degradar directamente el rendimiento general.	Se aíslan principalmente en una base de datos de reportes.
Disponibilidad	Una falla en la aplicación puede afectar todo el sistema.	Sigue existiendo este riesgo porque continúa siendo una única aplicación, pero puede mitigarse mediante múltiples instancias, balanceadores y health checks.
Aislamiento lógico de fallos	Bajo.	Mayor debido a la separación modular y de almacenamiento.
Aislamiento físico de fallos	Bajo.	Medio. Las bases pueden encontrarse en servidores diferentes, aunque la aplicación continúa siendo monolítica.
Infraestructura	Sencilla: aplicación + una BD.	Moderada: aplicación + bases especializadas y, opcionalmente, caché, workers y replicación.
Pruebas	Pueden ser difíciles por el acoplamiento entre funcionalidades.	Los límites entre módulos facilitan pruebas unitarias, de integración y pruebas específicas por dominio.
Integración con otros	Limitada.	Alta mediante API REST.
Evolución futura	Difícil debido a las dependencias existentes.	Alta. Los módulos críticos pueden convertirse posteriormente en servicios independientes.

Especificación y Contrato OpenAPI 3.0 (Swagger)

API REST - Sistema de Votaciones 1.0.0 OEA 3.0

API REST para la gestión del Sistema de Votaciones. Permite autenticar usuarios, administrar votaciones y candidatos, registrar resultados y consultar resultados electorales.

Servidores			
http://localhost/sistema_votaciones/api/v1 - Servidor local de desarrollo		Autorizar	
Autenticación		Operaciones de autenticación	▼
Votaciones		Gestión de procesos electorales	▼
Candidatos		Gestión de candidatos	▼
Resultados		Registro y consulta de resultados	▼

Autenticacion		Operaciones de autenticación	^
POST	/auth/login	Iniciar sesión	▼
Votaciones		Gestión de procesos electorales	^
GET	/votaciones	Listar votaciones	▼
POST	/votaciones	Crear una votación	▼
GET	/votaciones/{votacionId}	Obtener una votación por ID	▼
PUT	/votaciones/{votacionId}	Actualizar completamente una votación	▼
PATCH	/votaciones/{votacionId}	Actualizar parcialmente una votación	▼
DELETE	/votaciones/{votacionId}	Eliminar una votación	▼

Candidatos Gestión de candidatos ^		
GET	/votaciones/{votacionId}/candidatos	Listar candidatos de una votación 🔒 ▼
POST	/votaciones/{votacionId}/candidatos	Registrar un candidato en una votación 🔒 ▼
Resultados Registro y consulta de resultados ^		
POST	/resultados	Registrar resultado de una mesa 🔒 ▼
GET	/votaciones/{votacionId}/resultados	Consultar resultados de una votación 🔒 ▼

Códigos de estado HTTP

Códigos de éxito	Códigos de error
200 – Operación realizada correctamente.	400 Bad Request – Datos enviados incorrectos.
201 – Recurso creado.	401 Unauthorized – Recurso no encontrado.
	404 Not Found – Recurso solicitado no encontrado.
	500 Internal Server Error – Error interno del servidor.

Seguridad

La seguridad de la API REST se basa principalmente en un esquema de autenticación mediante de JWT, este permite validar la identidad de los usuarios antes de permitirles acceder a los recursos protegidos del Sistema de Gestión Electoral.

Procesos de autenticación

El usuario debe iniciar sesión enviando sus credenciales al endpoint:

POST /api/v1/auth/login

Example Value | Schema

```
{
  "usuario": "digitador01",
  "password": "MiClaveSegura123!"
}
```

El modulo de autenticacion e identidad valida las credenciales utilizando la información almacenada en Base de datos de identidad.

Si las credenciales son correctas el sistema genera un token JWT y lo devuelve al cliente.

Code	Description	Links
200	Autenticación exitosa	No links

Media type

application/json ▾

Controls Accept header.

Example Value | Schema

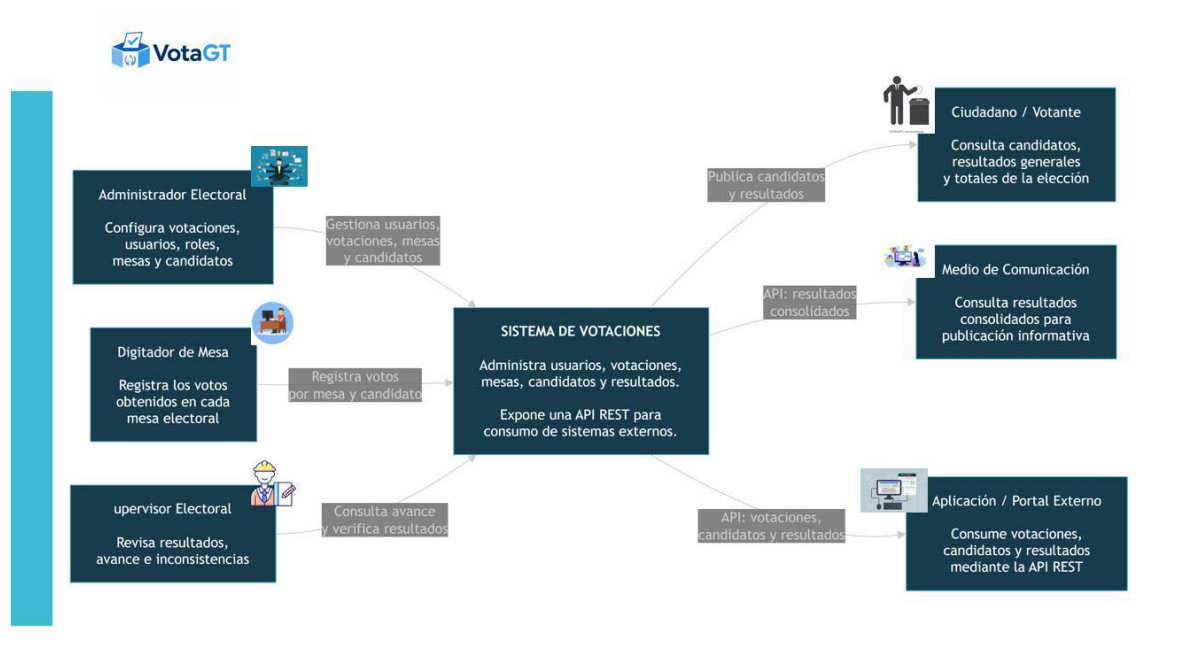
```
{
  "accessToken": "eyJhbGciOiJIUzI1NiJ9.eyJ1c2Vybm91dCI6ImRpdGItYXNjaWkiLCJ0eXBlIjoiYXNjaWkiLCJleDp5eXN": "eyJhbGciOiJIUzI1NiJ9.eyJ1c2Vybm91dCI6ImRpdGItYXNjaWkiLCJ0eXBlIjoiYXNjaWkiLCJleDp5eXN",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "usuario": {
    "id": 15,
    "usuario": "digitador01",
    "rol": "DIGITADOR"
  }
}
```

Autorización mediante roles y permisos

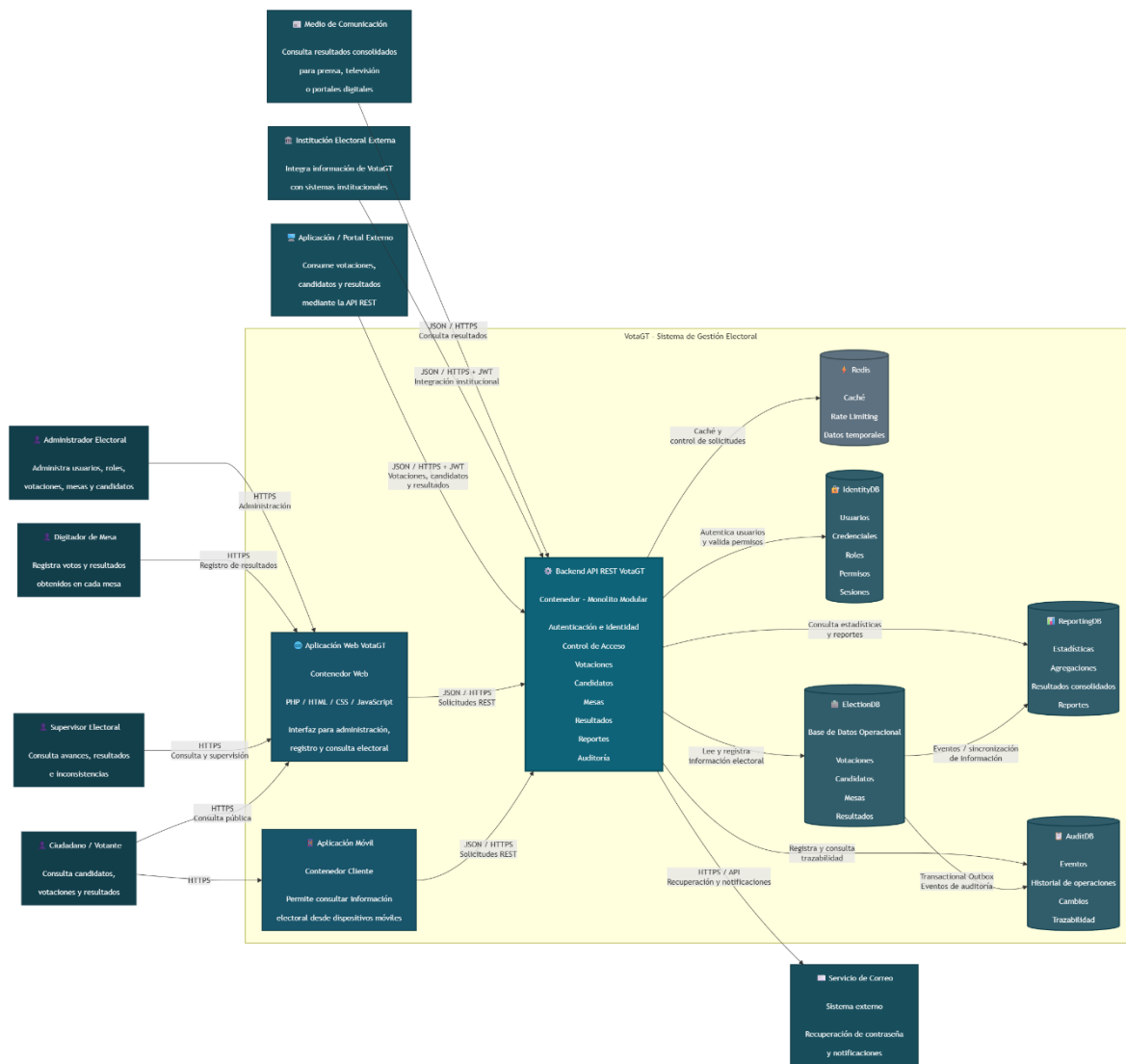
Adicional de la autenticación de los usuarios, el sistema controla las operaciones que cada uno puede realizar. Esto se gestiona mediante roles y permisos almacenados en Base de datos de identidad.

Planos Técnicos de Arquitectura bajo el Modelo C4

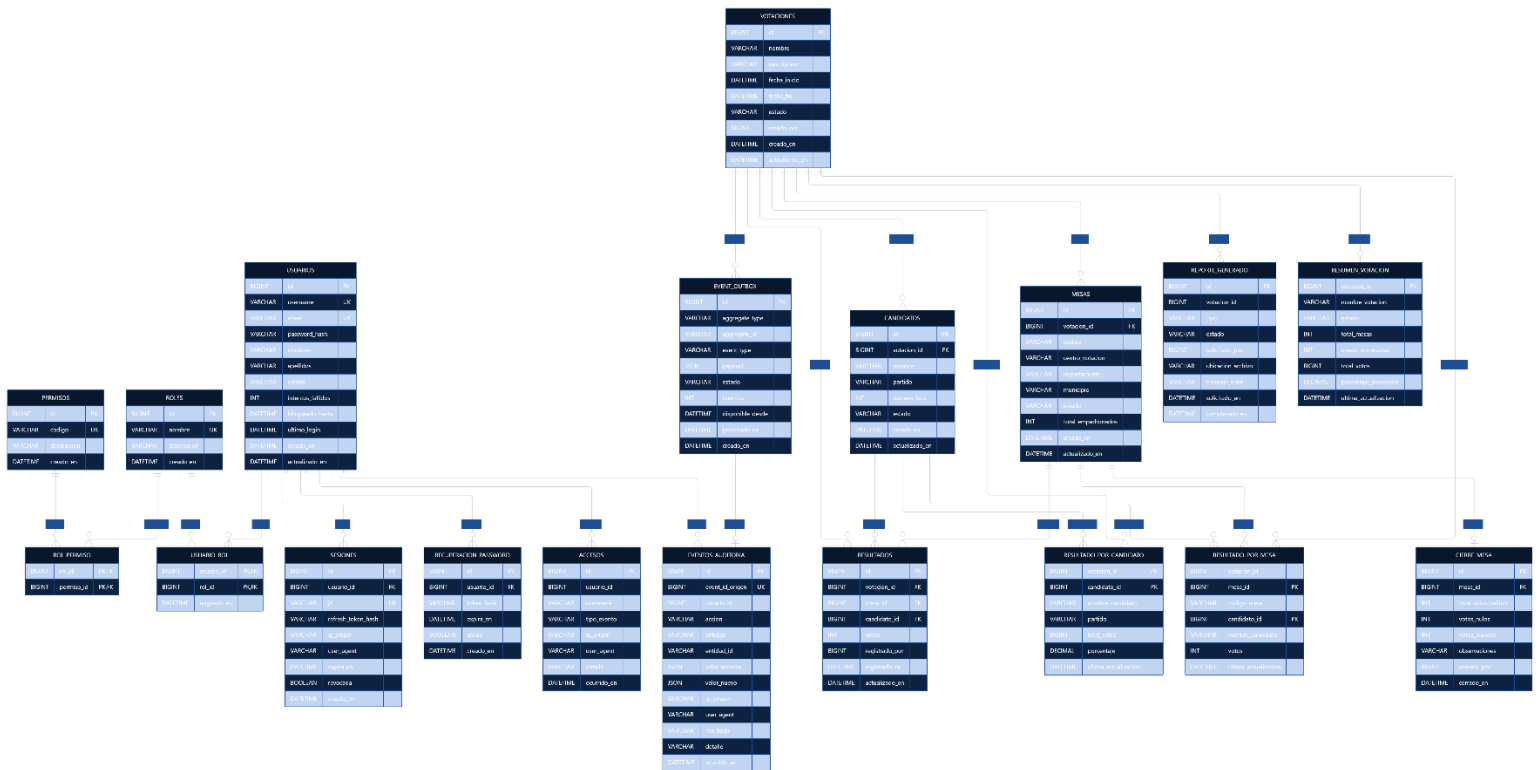
Nivel 1 – Diagrama de contexto



Nivel 2 – Diagrama de contenedores



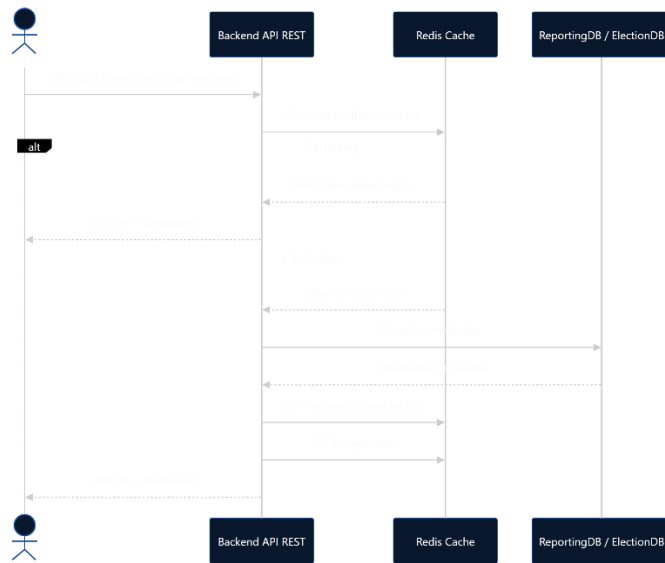
EDR



Estrategia de Caché

El sistema de gestión electoral utiliza Redis como mecanismo de cache para reducir la cantidad de consultas repetitivas realizadas sobre la base de datos de elecciones y la base de datos de reporte, el patrón utilizado es **CACHE-ASIDE** ya que permite que la aplicación controle cuando consultar, almacenar e invalidar información en Redis manteniendo las bases de datos como fuente oficial de información.

Cuando un cliente realiza una consulta, el Backend API REST verifica primero si la información se encuentra almacenada y si existe devuelve desde el caché



Claves de Caché y TTL

Las claves de Redis siguen una estructura descriptiva que permite identificar fácilmente qué información contienen.

Cache Key	Información	TTL recomendado
votacion:{id}	Información general de una votación	300 segundos
votacion:{id}:candidatos	Candidatos asociados a una votación	600 segundos
votacion:{id}:resultados	Resultados consolidados	15–30 segundos durante votación activa
votacion:{id}:resultados	Resultados consolidados de votación cerrada	300–900 segundos
votacion:{id}:resumen	Estadísticas generales	30–60 segundos
mesa:{id}:resultados	Resultados correspondientes a una mesa	30–60 segundos

Cache Key	Información	TTL recomendado
usuario:{id};permisos	Roles y permisos resueltos	300 segundos
ratelimit:{usuario}:{endpoint}	Contador de solicitudes	60 segundos

Política de invalidación ante mutaciones

El TTL por sí solo no es suficiente.

Cuando una operación modifica información importante, la caché relacionada debe invalidarse para evitar entregar información obsoleta.

Por ejemplo:

- POST /api/v1/resultados

puede modificar los resultados de:

- Votación 15
- Mesa 120

En ese momento deben invalidarse claves como:

- votacion:15: resultados
- votacion:15: resumen
- mesa:120: resultados

En Redis la operación conceptual sería:

- DEL votacion:15: resultados
- DEL votacion:15: resumen
- DEL mesa:120: resultados

La próxima vez que un cliente consulte:

- GET votacion:15:resultados

Redis responderá:

- Cache Miss

por lo que la API consultará nuevamente la base de datos y almacenará información actualizada.

